which the `FirstName` property has a `[Display]` attribute, but the `Email` property doesn't:

```
public class UserModel
{
    [Display(Name = "Your name")]
    public string FirstName { get; set; }
    public string Email { get; set; }
}
```

The following Razor

```
<label asp-for="FirstName"></label>
<label asp-for="Email"></label>
```

would generate this HTML:

```
<label for="FirstName">Your name</label>
<label for="Email">Email</label>
```

The inner text inside the `<label>` element uses the value set in the `[Display]` attribute, or the property name in the case of the `Email` property. Also note that the `for` attribute has been generated with the name of the property. This is a key bonus of using Tag Helpers; it hooks in with the element IDs generated by other Tag Helpers, as you'll see shortly.

NOTE The `for` attribute is important for accessibility. It specifies the ID of the element to which the label refers. This is important for users who are using a screen reader, for example, as they can tell what property a form field relates to.

As well as properties on the `PageModel`, you can also reference sub-properties on child objects. For example, as I described in chapter 16, it's common to create a nested class in a Razor Page, expose that as a property, and decorate it with the `[BindProperty]` attribute: